Tools, learning, and protocols

Architecture 2: University teaching deck

How does a model that only produces text come to act on the world, and how are tools made reliable and reusable?

Tool creation, definition, selection, calling, output processing, tool learning, native tool calling, MCP, and Skills.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Using tools

Explain why an LLM can only express the intention to use a tool. Describe the five steps: creation, definition, selection, calling, and output processing.

Learning tools

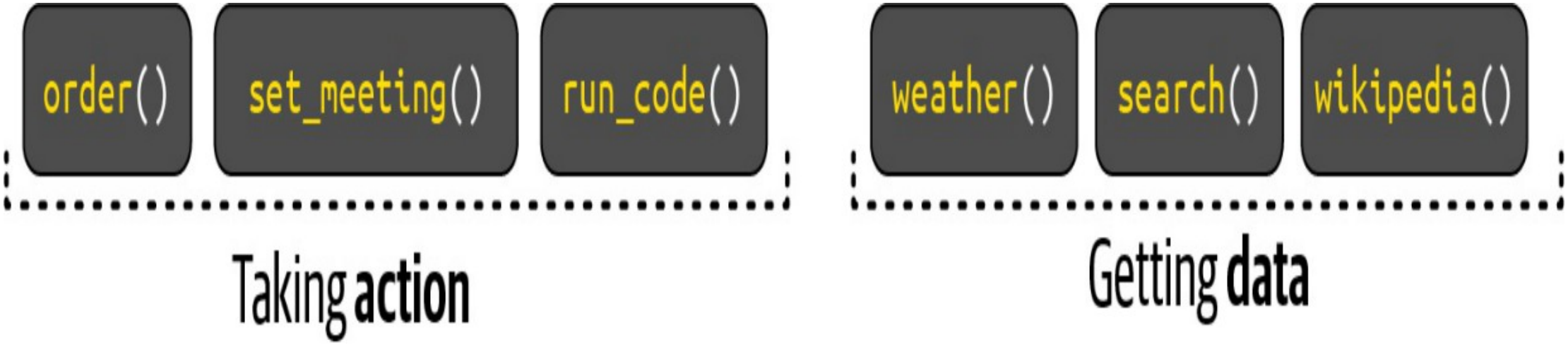
Compare in-context learning, supervised fine-tuning (Toolformer), and reinforcement learning (ToolRL, Search-R1). Explain native tool calling and chat templates.

Standards and procedures

Explain the N × M problem that the Model Context Protocol solves, its host, client, and server roles, and how Skills add procedural knowledge through progressive disclosure.

Source: Ch. 5, PDF pp. 239–296

Tools let an LLM act or retrieve



A function of text

On its own an LLM is a function that takes text and returns text. That supports chatbots but not interaction with the environment.

Two kinds of tool

Tools either take an action (order, schedule a meeting, run code) or get data (weather, search, an encyclopedia). They can also give access to memory or to specialized models.

Think but not act

Without tools, an agent can think and plan on its own but cannot act on its own. It can communicate the intention of searching the web or using a calculator, but it cannot do it.

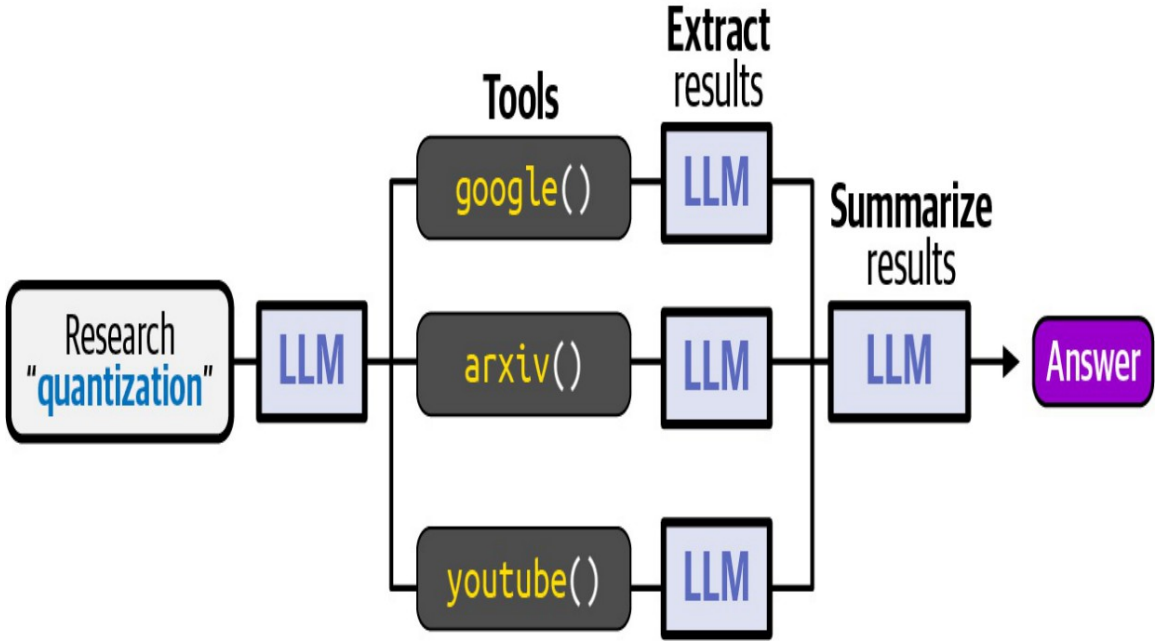
Grootendorst & Alammar (2026). Fig. 5-1, PDF p. 239

A fixed flow calls tools in a set order

No autonomy
When the flow is fixed, the developer decides which tools run and in what order. The model does not choose.

Example
A research workflow always calls arXiv, a web search, and a video search, summarizes each result, and then summarizes the summaries.

When it fits
A predetermined sequence suits routine, well-specified tasks where the same tools are always needed.



Grootendorst & Alammar (2026). Fig. 5-2, PDF p. 240

An agent chooses tools dynamically

Autonomous selection

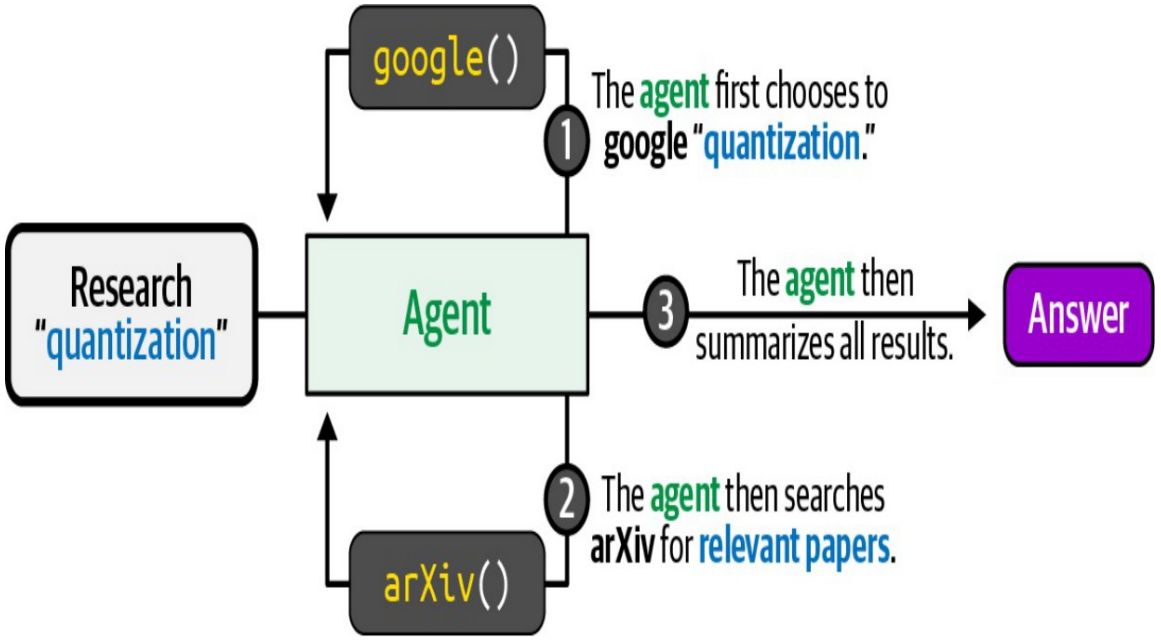
With greater autonomy, the agent decides which tool to use and when. The figure is still a sequence of LLM calls, but the agent picks the tools.

Example sequence

The agent first searches with a web tool, then decides to search arXiv for relevant papers, and finally generates an answer informed by both searches.

Open questions

How are tools created, how is their output processed, how are tools selected, and how many can a model handle? The rest of the chapter addresses these.



Grootendorst & Alammar (2026). Fig. 5-3, PDF p. 240

Tools connect the agent to its environment

Main interface

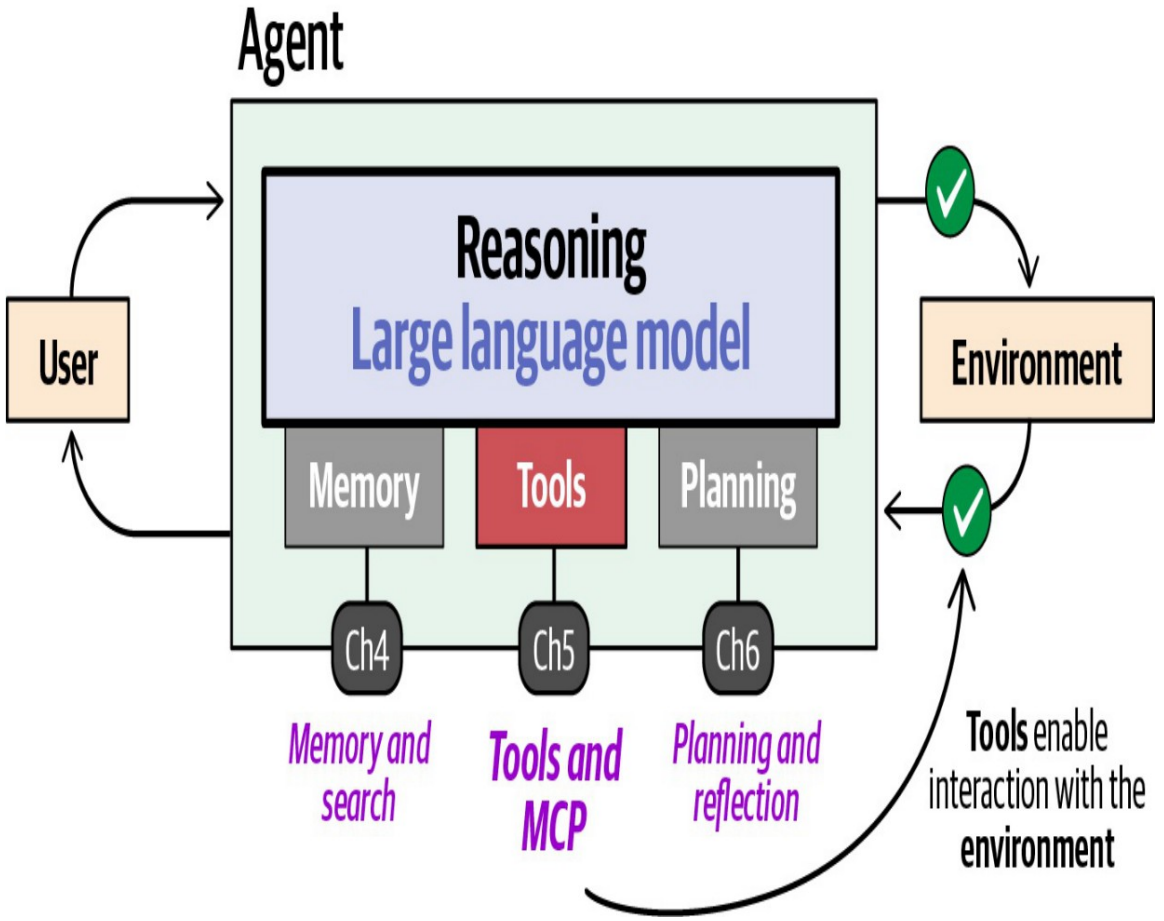
A collection of tools is the main component that allows the agent to interact with its environment, as the green checks in the figure show.

Roadmap link

The figure labels the three added modules by chapter: memory and search (Chapter 4), tools and MCP (Chapter 5), and planning and reflection (Chapter 6).

Why before planning

Planning would be a theoretical exercise without tools. An LLM cannot act autonomously if it cannot act at all.



Grootendorst & Alammar (2026). Fig. 5-4, PDF p. 241

Tool usage has five steps

The five steps

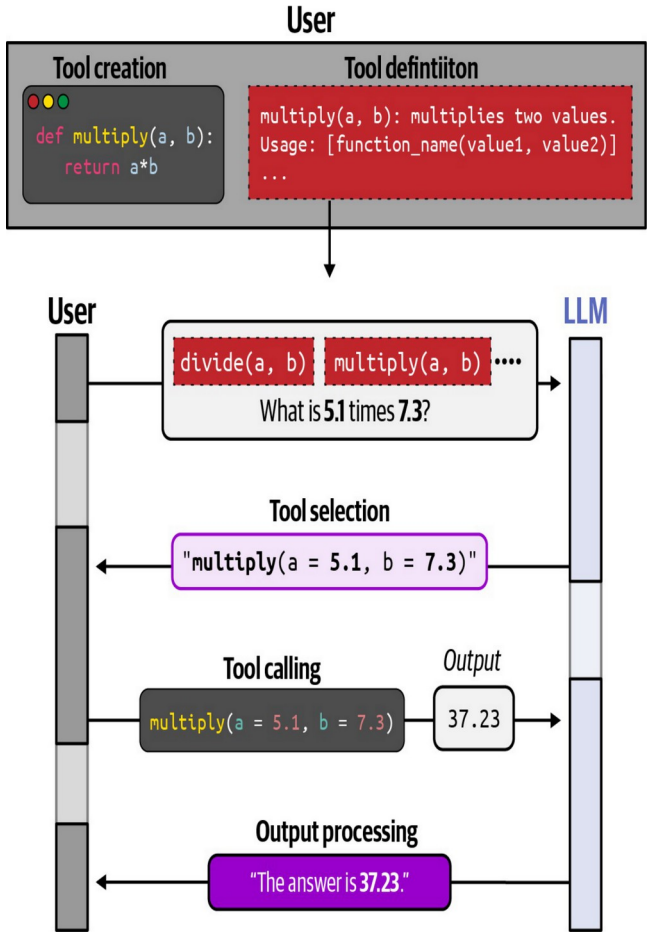
Tool creation, tool definition, tool selection, tool calling, and output processing. The figure traces one question through all five.

Intention only

The LLM does not call the tool. It shows the intention of doing so, and an external, usually automated, process executes the call.

Adapted flow

The figure adapts OpenAI's tool calling flow, adding the creation step and highlighting what a single tool call involves.



Grootendorst & Alammar (2026). Fig. 5-5, PDF p. 243

Creating a tool for an LLM

A tool is a function

Tools are usually functions reached through an API or internal code. The example multiplies two values, because LLMs are not known for their mathematical ability.

Text in, number out

The example takes its two inputs as strings and converts them to numbers, because LLMs return only text that has to be converted.

A registry

The simplest way to find a function by name is to keep it in a dictionary. The book's Tools class stores each tool's name, function, and description, and can list which tools need approval.

Design for the model

Ask whether the user, here the LLM, understands what the tool does, what it returns, and what its variable names mean. Good documentation matters because the model must be told what exists.

Grootendorst & Alammar (2026). Code, PDF pp. 244–245

Defining tools: learning, prompts, and schemas

Why define

Without being told which tools exist, what they do, how they are used, and in which context, the LLM cannot use them properly.

Learning

Specific tools can be learned during fine-tuning, but that is costly and limits how many tools can be added. Recent models instead learn to recognize tool definitions in the prompt and use them dynamically.

Natural language

For a small set of tools, the system prompt lists them and states how to write a call. The format is arbitrary; a small parser must only recognize it, which makes this approach error-prone.

Prompt generation

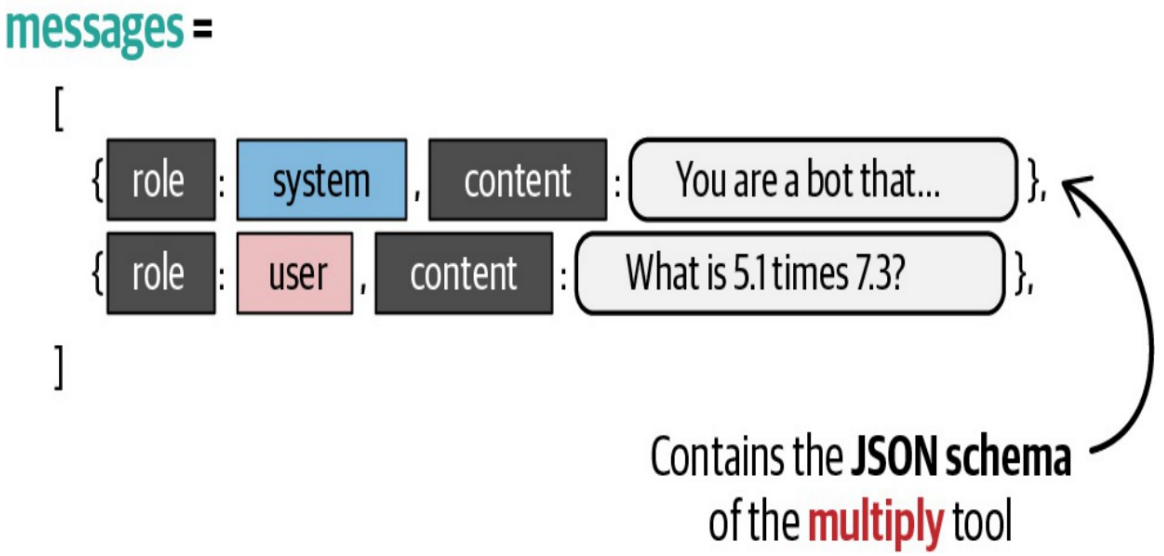
The Tools class can build this system prompt from the registry: a list of tool descriptions plus an instruction to reply with a JSON object naming the tool and its arguments.

Structured schemas define each tool

JSON schema
A tool can be described by a JSON Schema giving its name, description, and parameters with types, required fields, and ranges.

Passed as metadata
APIs often accept schemas as a separate parameter, such as OpenAI's tools parameter. In practice they are typically processed like regular prompt text, for example in the system prompt.

Still interpretation
The model still has to decide how and when to use the tool. The clearer and more descriptive the schema, the more likely the right decision.



Grootendorst & Alammar (2026). Fig. 5-6, PDF p. 249

Best practices for defining tools

Document the tool

Include extensive descriptions so the agent understands what the tool can do. Documentation is often neglected in ordinary development but is essential here.

Minimize the number

Many tools expand capability but make selection harder. There is no gold standard; fewer than ten tools is a good start, and selection methods matter more as the number grows.

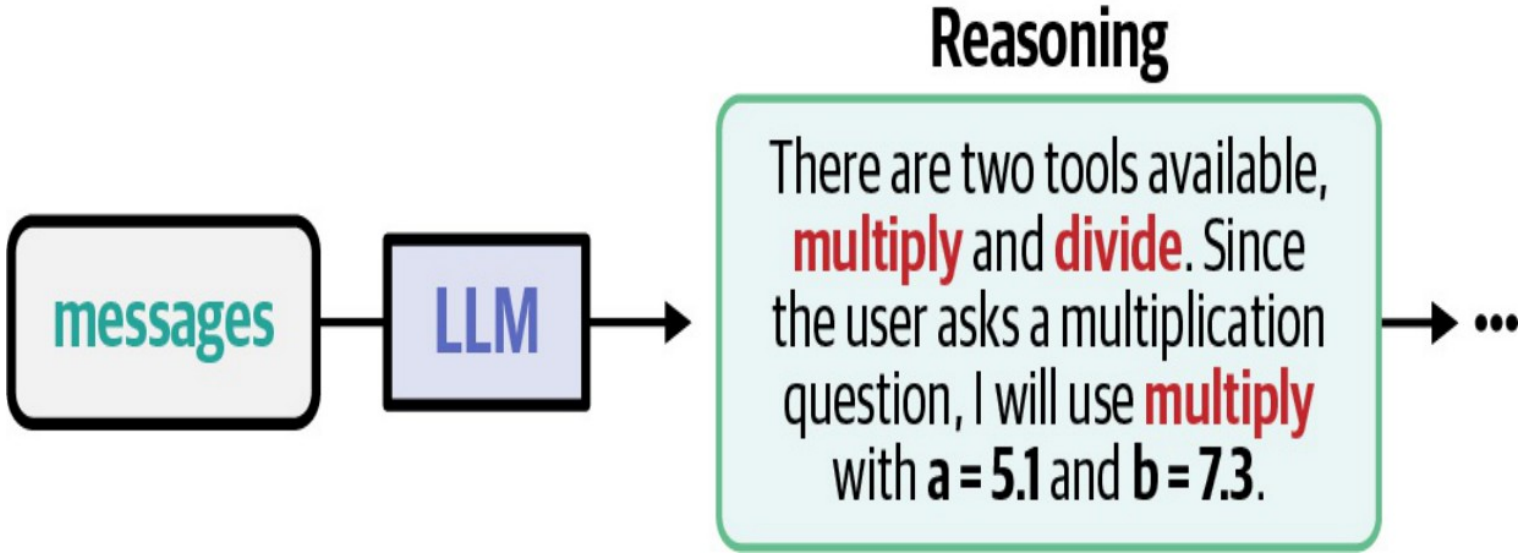
Minimize the scope

Complex tools with many parameters are hard to use, even for people, and especially for smaller models. Prefer small, focused tools.

Keep the definition

Tool definitions in the system prompt must persist. The summarizing memory from Chapter 4 overwrites the system prompt, so it would erase the tool definitions unless only one section is updated.

Tool selection relies on reasoning



Choosing and using
With dozens of complex tools, the LLM must choose the most appropriate one, or none, and then use it correctly. Following an extensive schema takes a capable model.

Reasoning models help
Reasoning LLMs can spend tokens thinking about which tool to use and how, as long as the thinking fits in the context window and leaves room for the answer.

Planning matters
For a simple question such as 1 + 1, selection is easy. Planning a vacation with a budget needs several steps and an agent that plans its behavior first.

Grootendorst & Alammar (2026). Fig. 5-7, PDF p. 250

Discovering tools with retrieval

Too many schemas

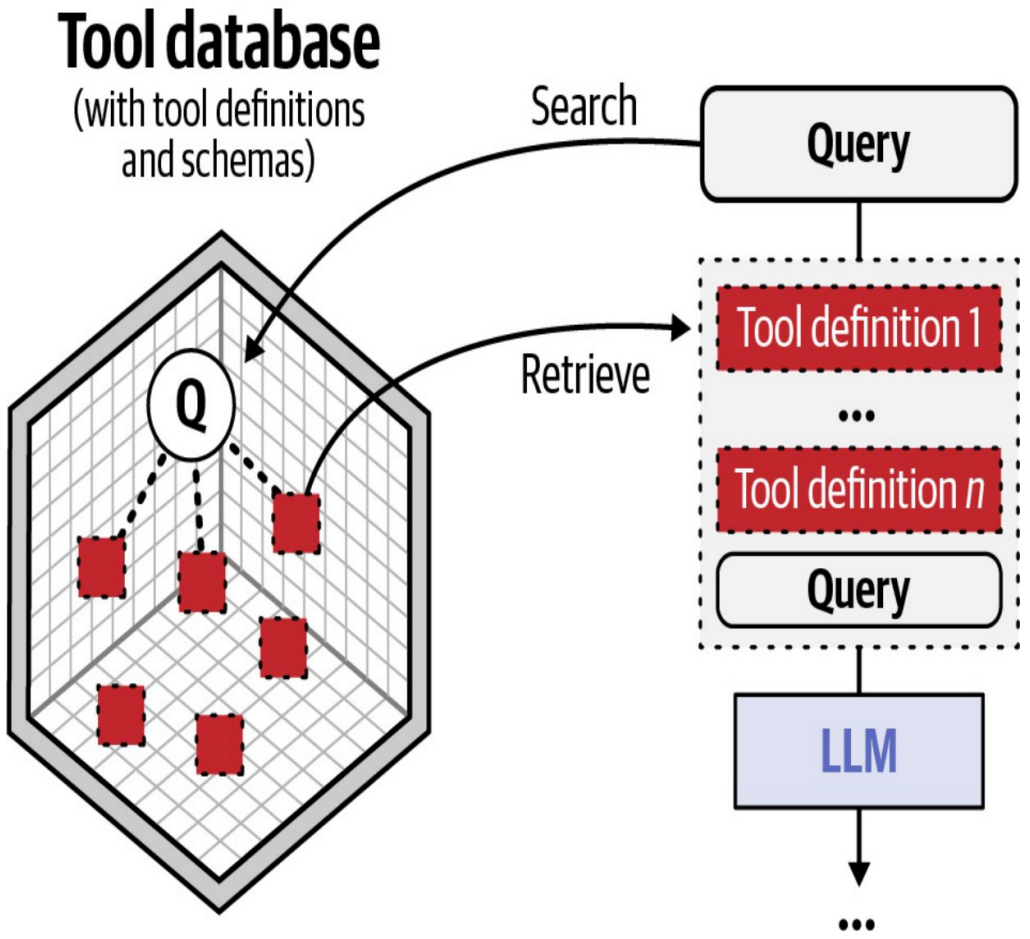
As in context engineering, filling the window with tool schemas decreases performance, so discovering relevant tools becomes important as their number grows.

RAG for tools

All tool definitions and schemas can be stored in a separate database. The query retrieves only the definitions most relevant to the request.

Same ideas

This applies the retrieval ideas from Chapter 4 to tool definitions instead of documents.



Grootendorst & Alammar (2026). Fig. 5-8, PDF p. 251

Tool calling: the answer is only an intention

A string

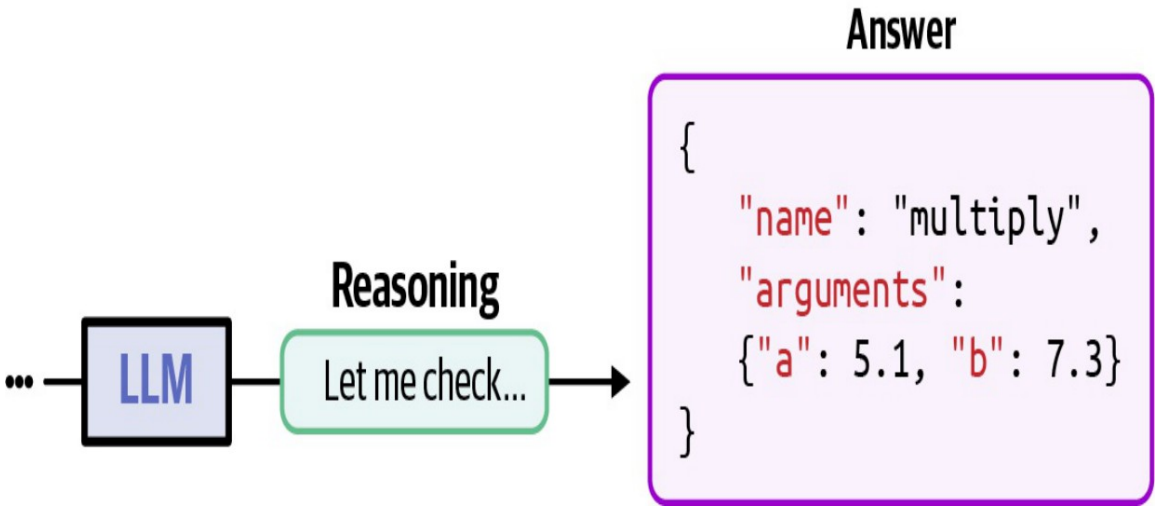
When the model finishes thinking, it outputs a string. If it is capable enough, the string contains a correctly formatted call with the arguments.

No execution

That string is merely the intention to call a tool. Without help from the user or additional software, nothing happens.

Extending the Tools class

To act on it, the Tools class needs functions that extract and parse the JSON call and then execute it.



Grootendorst & Alammar (2026). Fig. 5-9, PDF p. 252

Parsing and executing the call

Parse

The parse function looks for a tool call in the model's text, extracts the JSON, and stores it as a structured tool call in the response.

Execute

The execute function reads the tool name and arguments, looks the tool up in the registry, and runs it. In the example, multiplying 3.1 by 6.5 returned about 20.15.

Human in the loop

Before running a tool marked as needing approval, execution asks the user to allow it. If the user refuses, the tool is not run and the model is told it was denied.

More robust parsing

Simple text matching fails when the structure differs slightly. Packages such as jsonschema and Pydantic validate the structure and can also generate schemas for the model.

Tool output returns as a message

The model cannot see

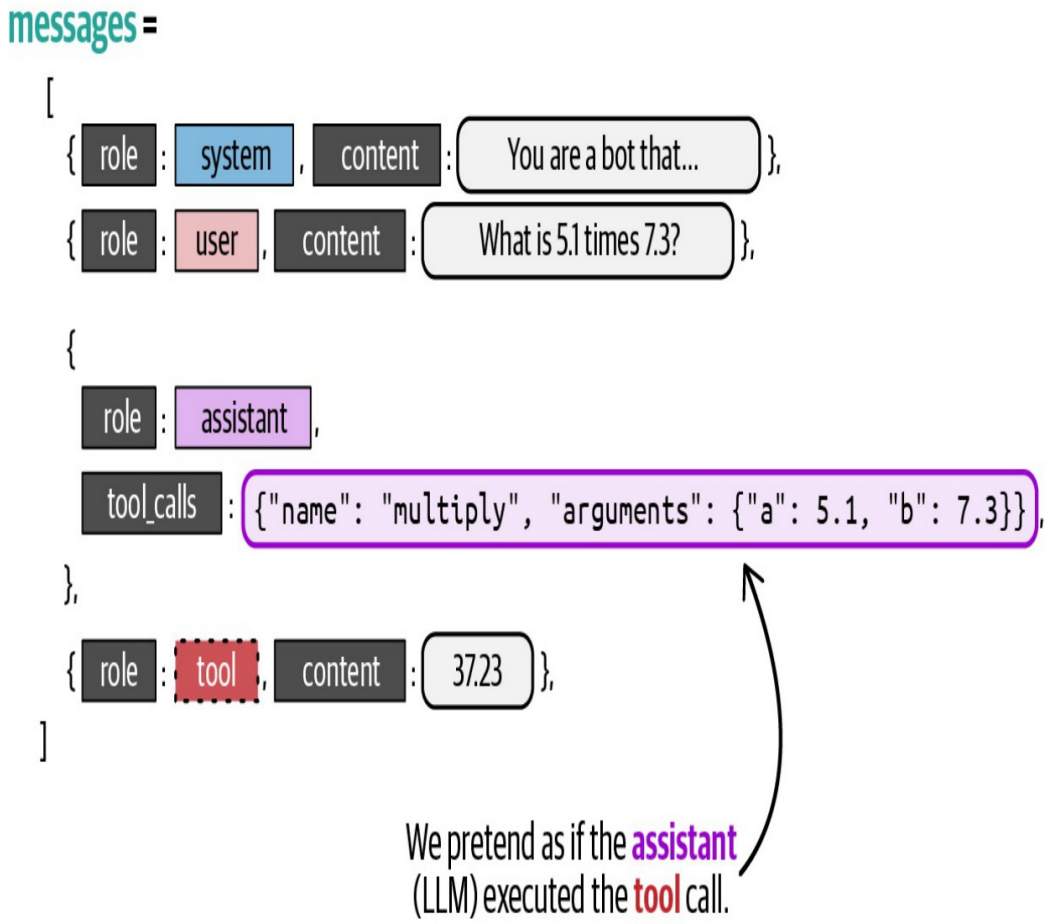
The tool call happens outside the model's view. To tell the model what happened, the messages are updated as if the model had made the call.

Two roles

An assistant message records the call, and a tool message returns the output. The tool role is used only by models trained for native tool calling.

Pretend execution

The assistant is shown as if it executed the tool, although the user or an automated system actually did.



Grootendorst & Alammar (2026). Fig. 5-10, PDF p. 255

Observations without a tool role

The user role

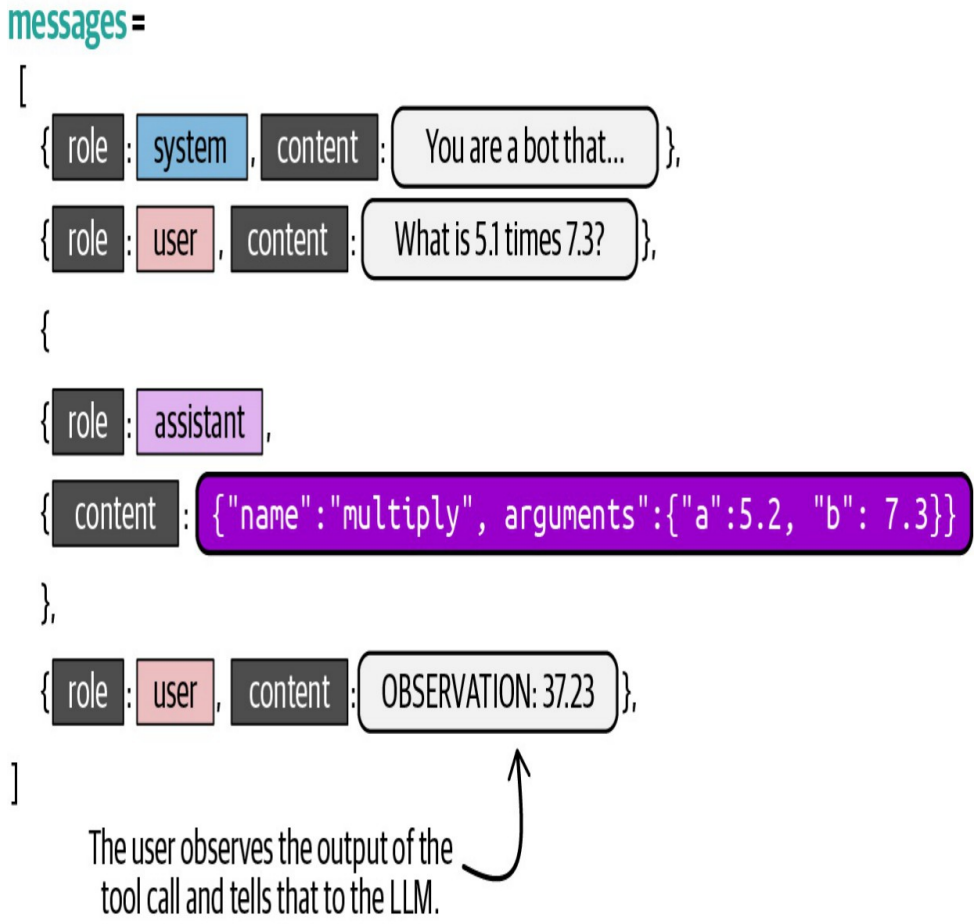
For models not trained with tool-calling tokens, the output is passed back using the user role, as if the user were telling the model the result.

OBSERVATION tag

An observation function returns the user role and the text OBSERVATION followed by the result. This tells the model that the call was made and what it returned.

Stopping

An is-done function stops the agent if there is no tool call or if the call is to a final-answer tool. Chapter 6 develops this stopping mechanism.



Grootendorst & Alammar (2026). Fig. 5-11, PDF p. 257

The LLM processes the tool's output

Why another call

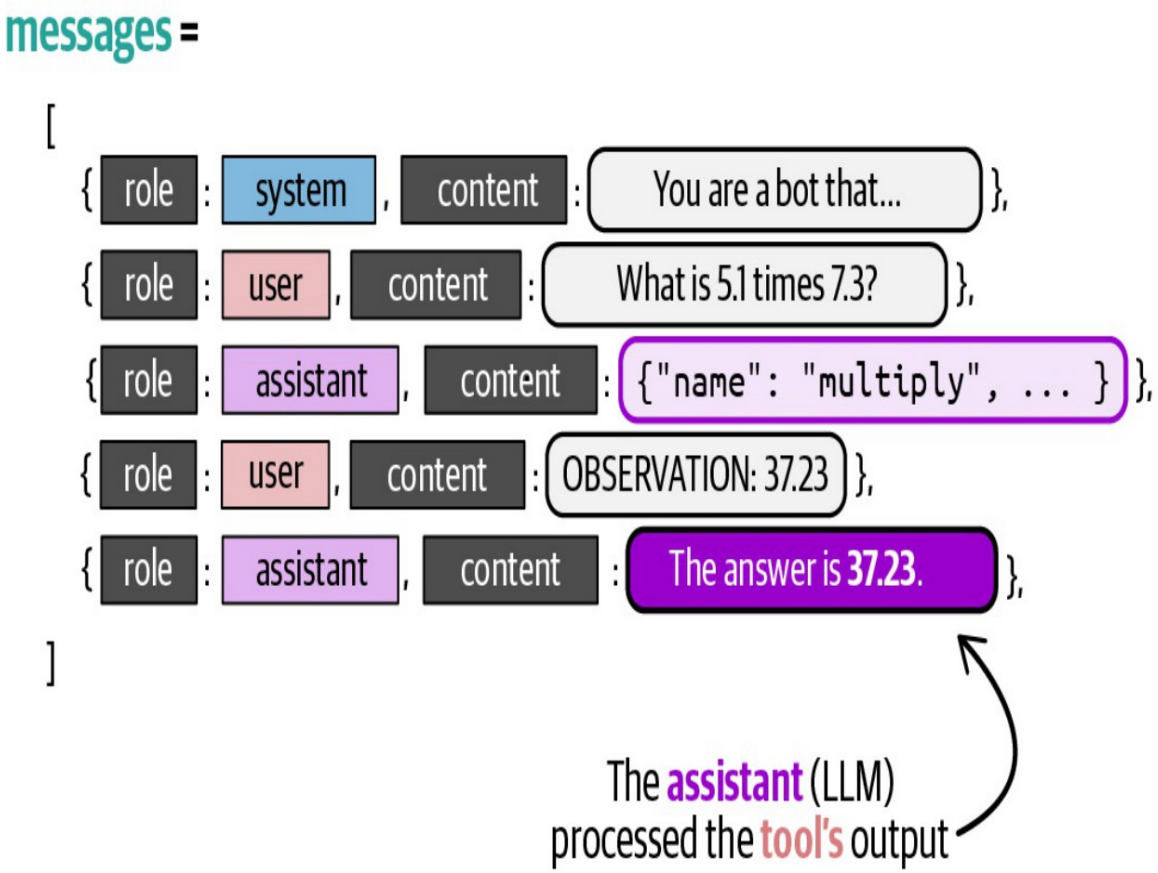
Although the tool gave the right output, the model may combine or summarize results, or present them in a nicer way according to pre-decided instructions.

History as the channel

The updated conversation history, including the observation, is sent back to the model for its final answer.

Final answer

In the figure the assistant answers that the result is 37.23, based on the observation it received.



Grootendorst & Alammar (2026). Fig. 5-12, PDF p. 258

Adding tools to the TinyAgent

System prompt

On creation, the agent builds a system prompt that includes the tool definitions and stores it in memory. Later chapters use the system prompt for more than tools.

A single step

The agent generates a response (the THOUGHT), parses any tool call, and stops if there is no call or if the call is the final answer. Otherwise it executes the action.

Execute an action

The tool is executed, and the result is added to memory as an OBSERVATION message. The trajectory records both the response and the observation.

ReAct labels

THOUGHT, ACTION, OBSERVATION, and ANSWER come from the Reason and Act framework that Chapter 6 uses to create autonomous behavior.

Checking that the tool was really used

The test

Asked what is 5.1 times 7.3, the agent returned OBSERVATION 37.23. The answer is correct, but was it calculated by the model or by the tool?

Inspect the history

The stored messages show the system prompt, the user's question, an assistant message with a JSON call to multiply, and a user message with the observation. The tool was used.

An orchestrator

Many steps must go right, so the LLM has to be an effective orchestrator. That depends on its reasoning ability and reliability, as explored in Chapter 3.

A more stable route

Training a model on tool-calling tasks tends to generalize better than prompt instructions alone. The next section covers how models learn this.

Grootendorst & Alammar (2026). Results, PDF pp. 261–262

Three ways models learn to use tools

In-context learning

Show a few examples of correct tool calls in the prompt, so the model imitates the format. It needs no training but uses context space and works better with more capable models.

Supervised fine-tuning

Train the model on examples of prompts paired with correct tool calls. It was popular in 2023 and early 2024 because it was effective and relatively cheap, but it generalizes less well.

Reinforcement learning

Let the model try, and reward correct tool use. This relies on trial and error and feedback instead of imitation, and tends to generalize better.

In-context learning with few-shot examples

Few-shot prompting

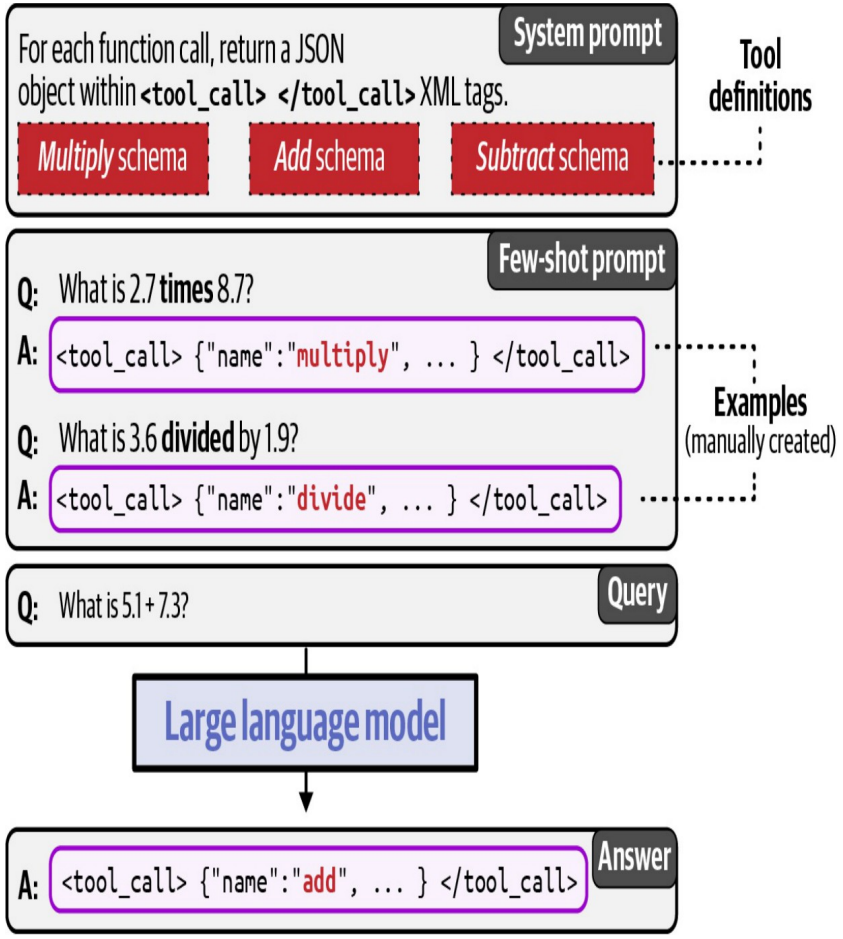
Show several examples of the behavior you want the model to repeat. Here the examples demonstrate how to output a tool call in a specific format.

Manual examples

The figure includes a system prompt with tool definitions and two hand-written examples, such as multiplying 2.7 by 8.7, before the real question.

No fine-tuning

This enables new behavior without changing the model's weights, as discussed in Chapter 3.



Grootendorst & Alammar (2026). Fig. 5-13, PDF p. 263

Examples can be written as message roles

Pretend history

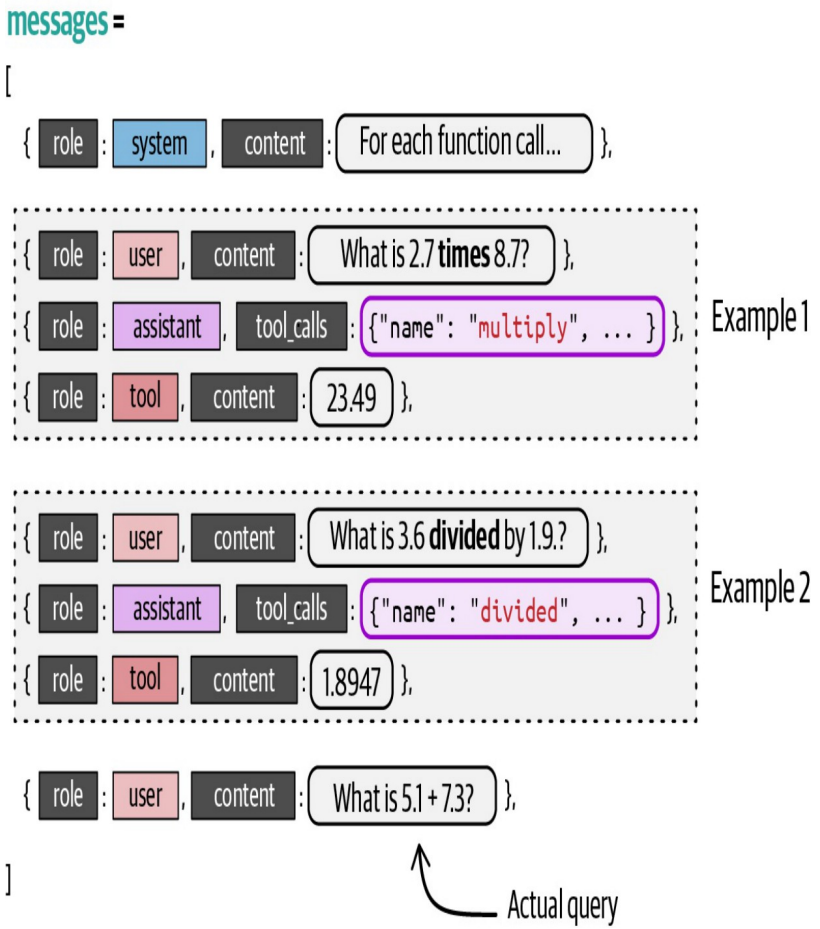
Since the messages structure already contains tool calls and tool results, the examples can be written as earlier turns instead of as text in the prompt.

Model believes it has done this

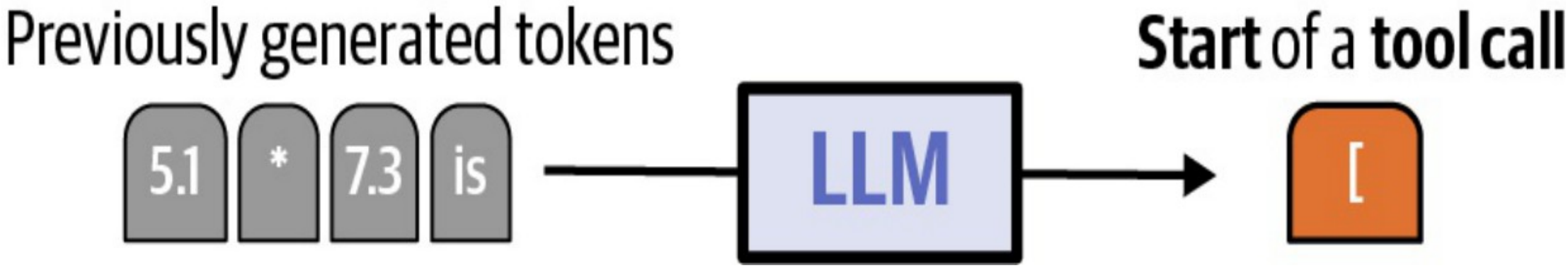
The model sees that it appears to have used the tools before, so it has examples of how to call them and what output to expect.

Pattern recognition

LLMs are good at recognizing patterns, so example calls help them follow the structure the developer had in mind.



Toolformer: fine-tuning for tool use



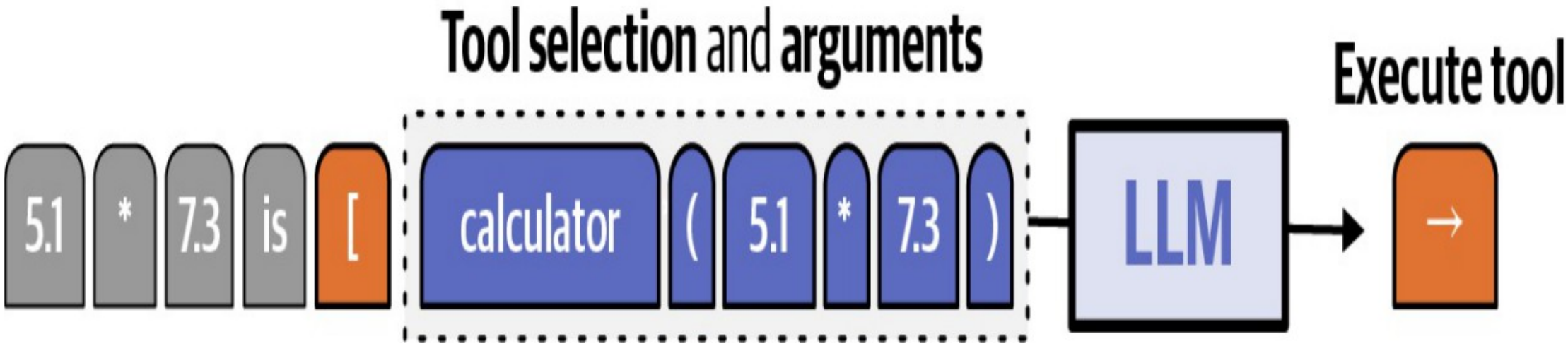
Why fine-tune
Prompting fills the context window and the model may ignore instructions if there are too many. Supervised fine-tuning builds tool-calling knowledge into the model.

In-line calls
Toolformer embeds the call and its output in the text it is generating, using the [and] tokens to mark where a tool call starts and ends.

The start token
Asked what is 5.1 times 7.3, the model generates tokens until it produces the [token, which signals the start of a tool call.

Grootendorst & Alammar (2026). Fig. 5-15, PDF p. 265

Toolformer selects the tool and arguments



After the bracket
The opening token tells the model to select a tool and generate its parameters in a fixed format.

Calculator call
In the example, the model writes the calculator tool and the expression 5.1 * 7.3.

The arrow token
The model continues until it generates an arrow token, which signals that it can stop generating and wait for the tool to be executed.

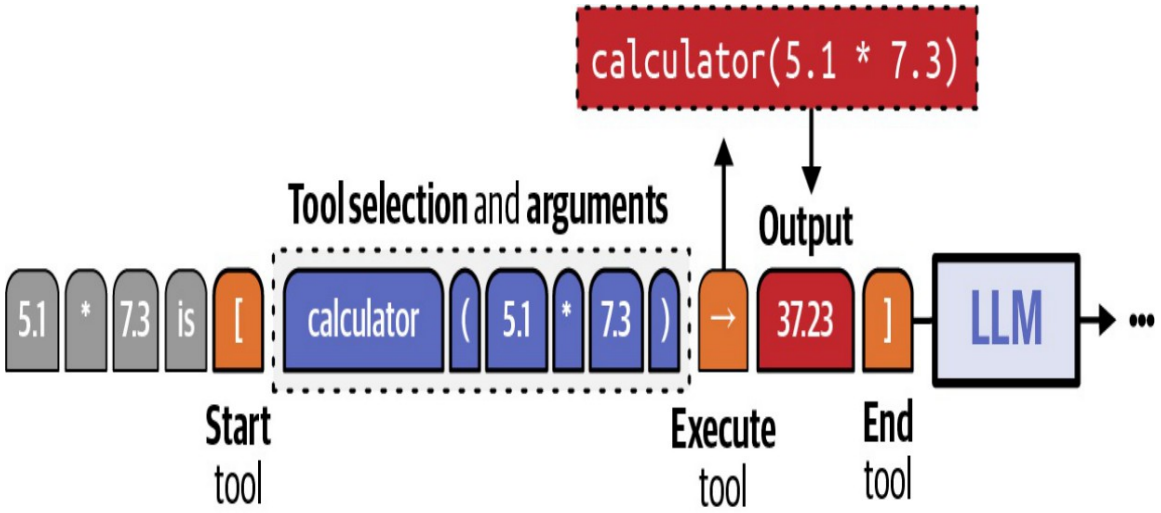
Grootendorst & Alammar (2026). Fig. 5-16, PDF p. 265

The tool's output is inserted as if generated

Executed outside
The user or an automated system runs the selected tool and obtains the result, here 37.23.

Added in line
Instead of using the messages structure, the output is appended to the already generated tokens, followed by the closing bracket token.

Continues generating
After the closing token, the tool call and its output are part of the generated text, and the model may keep generating if needed.



Grootendorst & Alammar (2026). Fig. 5-17, PDF p. 266

Toolformer end to end

Input and output

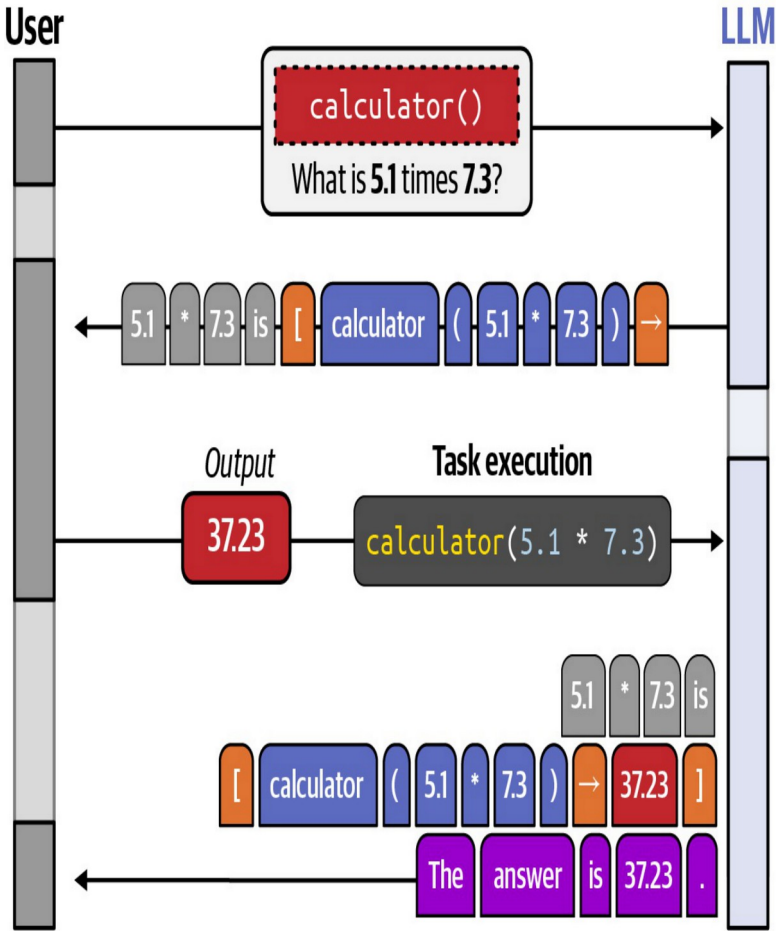
Input: what is 5.1 times 7.3? Output: 5.1 * 7.3 is, followed by the calculator call and result, then the answer is 37.23.

Fluent language

Embedding the result during generation lets the model use tools naturally and still produce fluent text.

Reasoning models

This suits reasoning LLMs, where tools are called during the reasoning process instead of passing messages back and forth, as with Search-o1 in Chapter 4.



Grootendorst & Alammar (2026). Fig. 5-18, PDF p. 267

How Toolformer's training data was made

Few-shot sampling

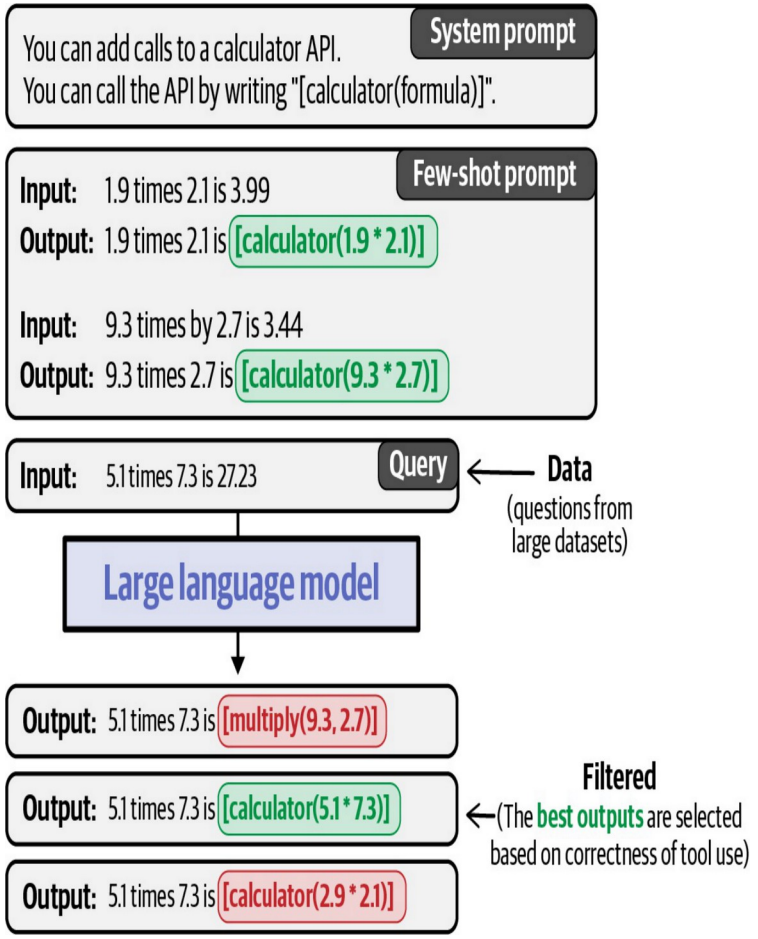
For each tool, a few-shot prompt was written by hand and used to sample many outputs that include tool calls.

Filtering

Outputs were filtered by the correctness of the tool use and its output and by whether the loss decreased, keeping the best.

Fine-tuning

The original data was adjusted to include tool calls, and GPT-J, a variant of GPT-3, was fine-tuned on it with supervised fine-tuning.



Grootendorst & Alammar (2026). Fig. 5-19, PDF p. 269

Why supervised fine-tuning for tools is limited

Sensitive to wording

SFT tries to recreate what it is shown, so it can be sensitive to the exact wording and prompt, which makes generalization difficult.

Many valid calls

For many prompts there is no single correct tool call. To read a README file, a model might call a get-readme tool or a get-contents tool, and both are valid.

Tools that never fail

Quality data often assumes that tools never fail, while real tools may fail, return partial errors, change their schemas, or behave inconsistently.

Why RL helps

These problems are less pronounced with reinforcement learning, which the next slides introduce as a more stable technique for generalizing tool use.

Reinforcement learning and tool-integrated reasoning

Trial and error

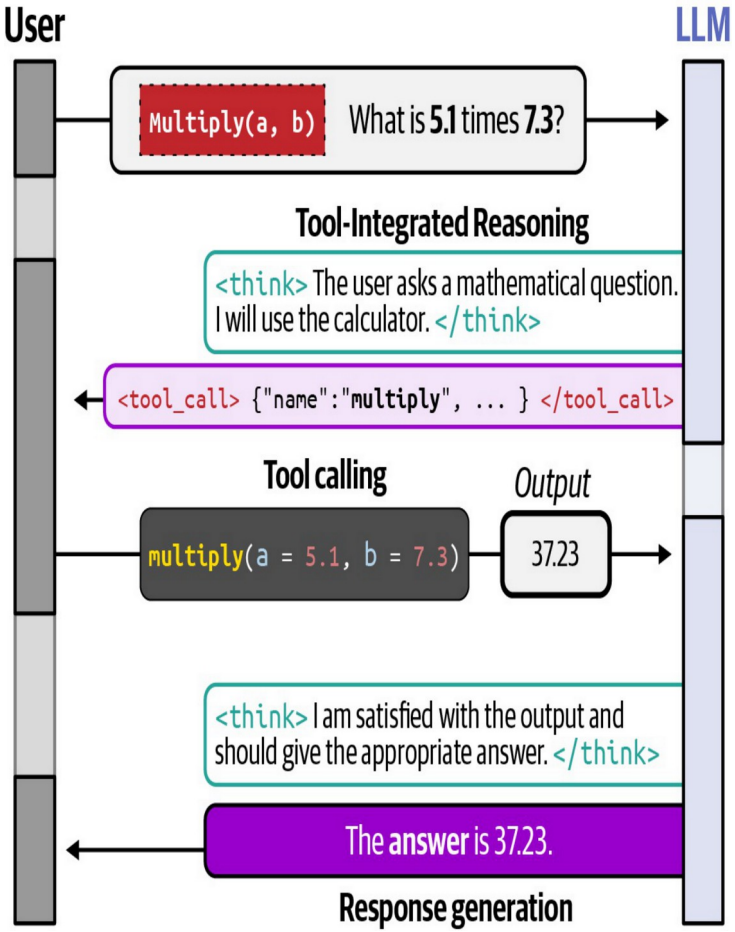
In RL a model develops better strategies by exploring feedback signals, rather than copying fixed examples with the correct answer.

Tool-integrated reasoning

Tool-learning is often built into the thinking process of reasoning models. This is Tool-Integrated Reasoning (TIR): tools inside the reasoning trace.

Several calls

A TIR trajectory may involve multiple tool calls, and the final answer depends on all intermediate calls and outputs.



Grootendorst & Alammar (2026). Fig. 5-20, PDF p. 271

ToolRL: rewards for correct tool calls

Method

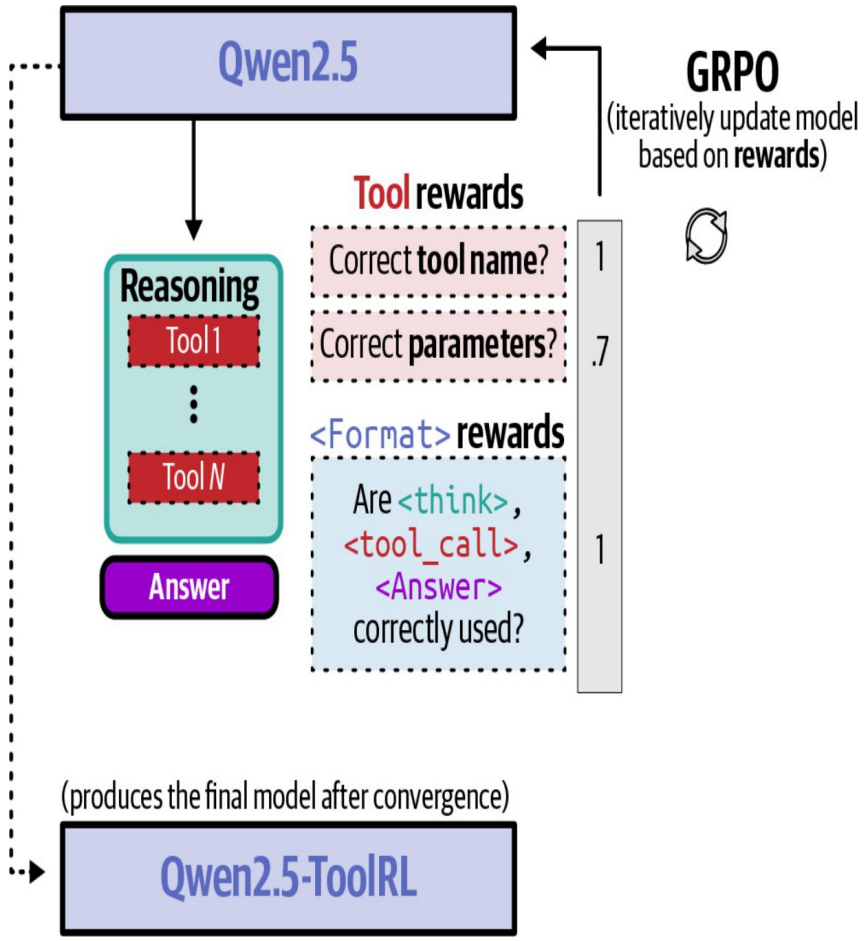
ToolRL uses GRPO, the algorithm from DeepSeek-R1. Special tokens separate thinking, tool calling, and answering.

Two rewards

A correctness reward checks whether the right tool names and parameters were used. A format reward is given if all required fields appear in the correct order.

Training data

About 4,000 sampled TIR traces were used to fine-tune Qwen2.5 models. Longer reasoning did not consistently help, and may harm smaller models.



Grootendorst & Alammar (2026). Fig. 5-21, PDF p. 273

Search-R1: the prompt template

A single tool

Search-R1 integrates one tool, search, into step-by-step reasoning. The model learns to produce one or more search queries autonomously.

Three parts

Reasoning is written between think tags, a search call between search tags, retrieved text between information tags, and the answer between answer tags.

Interleaving

Reasoning and searching can alternate several times before the answer. The approach was created as an open alternative to proprietary deep research systems.

> Answer the given question. You must conduct reasoning inside <think> and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <search> query </search>, and it will return the top searched results between <information> and <information>. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer> without detailed illustrations.

Grootendorst & Alammar (2026). Fig. 5-22, PDF p. 274

Search-R1: interleaved reasoning and search

Multi-turn search

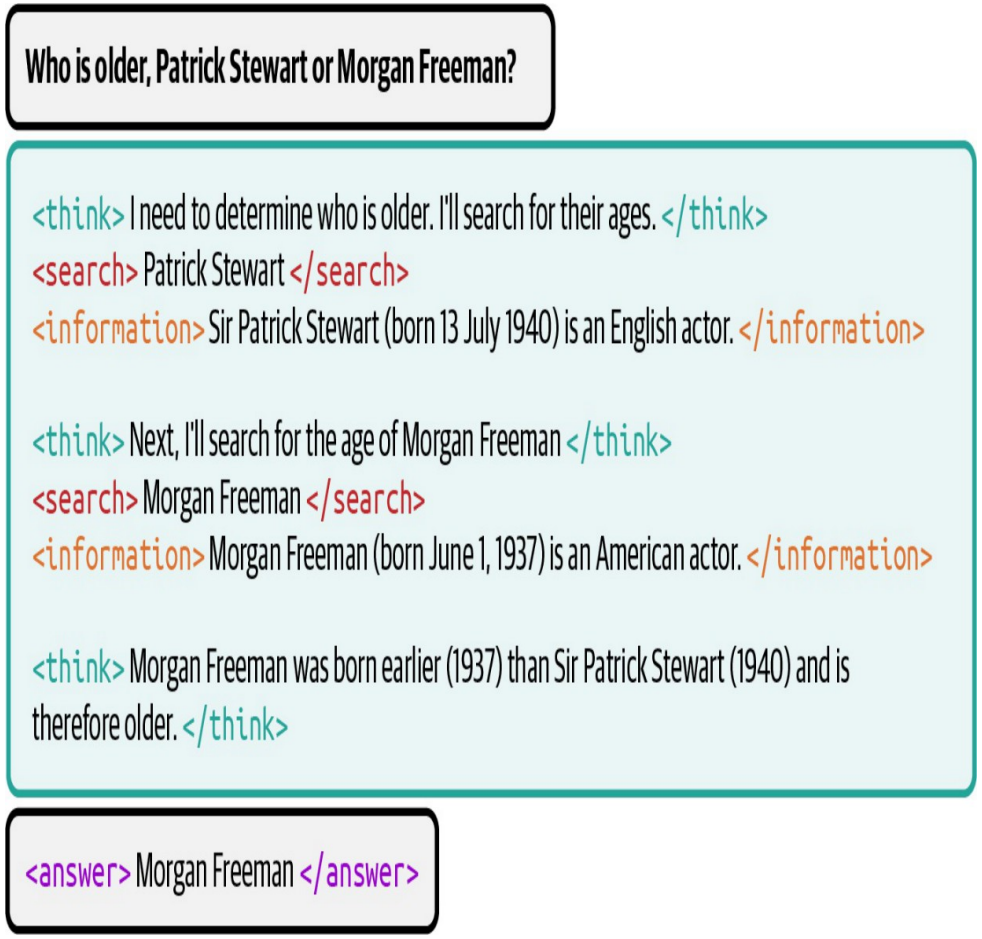
The result is tool-interleaved reasoning with several rounds of search, as in the figure where two actors' birth dates are looked up in turn.

Example

To decide who is older, Patrick Stewart or Morgan Freeman, the model searches each person, compares the dates, and then answers.

Outcome reward

Training used only accuracy rewards based on the task, with no format rewards, since the base model already follows structure well.



Grootendorst & Alammar (2026). Fig. 5-23, PDF p. 275

Search-R1: training and loss masking

GRPO training

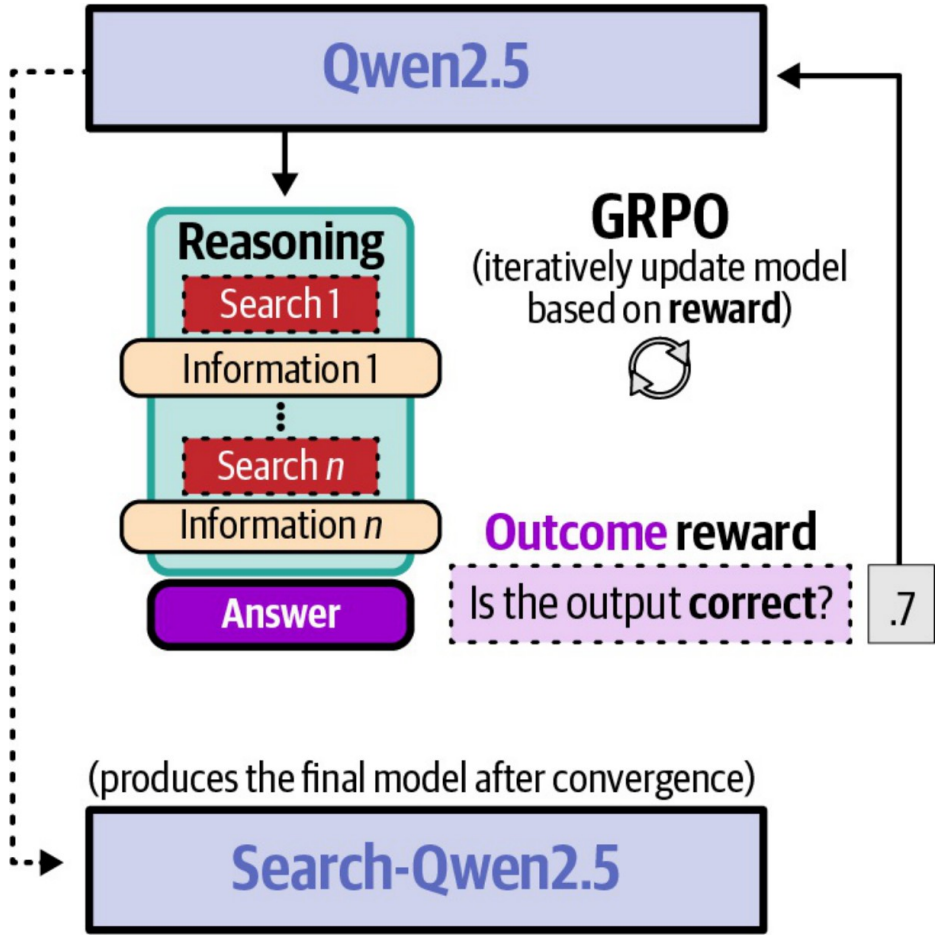
The authors explored GRPO among other algorithms to fine-tune the model, producing Search-Qwen2.5. The reward asks whether the output is correct.

Loss masking

GRPO losses are normally computed over all tokens. In Search-R1 the tokens returned by the search engine are masked, so the model is not trained to control text it did not generate.

Verifiable rewards

Reward structures for tool calling are straightforward, and RL works well for verifiable rewards such as coding and tool calls, as in Qwen3 and GPT-OSS.



Grootendorst & Alammar (2026). Fig. 5-24, PDF p. 276

SFT versus RL for tool use

What the model learns

SFT: to reproduce shown tool calls. RL: strategies that earn reward through trial and error, including how to use tool output in reasoning.

Data and signal

SFT needs curated demonstrations, which usually show only one path and assume success. RL needs a checkable reward, such as a correct tool name and arguments.

Generalization

SFT can be sensitive to the wording of prompts. RL tends to be more stable across tools and phrasings, according to the chapter.

Where it fits

SFT was an effective, inexpensive way to add tool calling in 2023 and early 2024. Many recent open models add RL with tool-based rewards.

Teaching addition. Concepts: Ch. 5, PDF pp. 262–276

Native tool calling uses the model's chat template

The hidden step

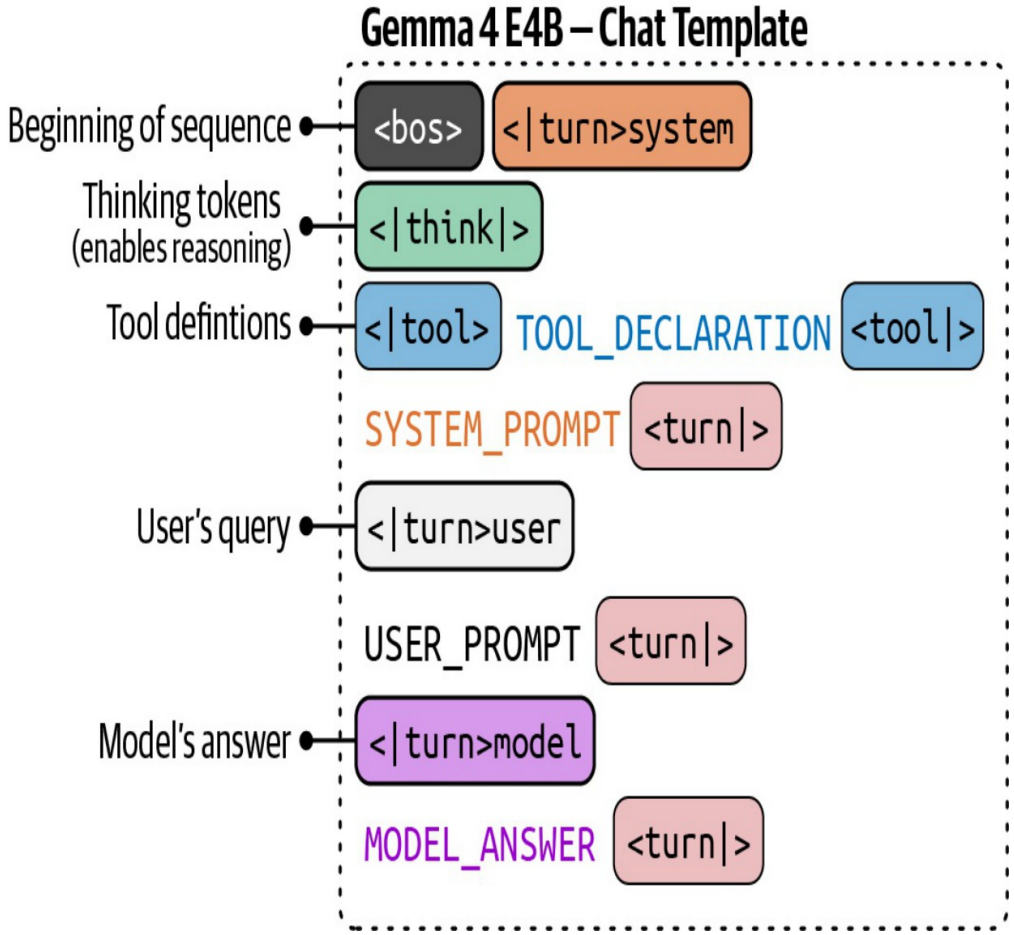
Inference engines such as Ollama or llama.cpp convert a standard way of describing tools into the chat template that the specific model was trained on.

Special tokens

Different models use different special tokens, for example for thinking. The template for Gemma 4 E4B marks system prompt, user query, and model answer with turn tokens.

Why uncover it

The conversion makes using tools easy but hides how it works. Looking at the template shows what the model actually sees.



Grootendorst & Alammar (2026). Fig. 5-25, PDF p. 278

A tool declaration in the model's own format

Standard schema in

The developer passes the JSON schema of the tool, the same standard format described earlier in the chapter.

Model format out

The inference provider translates the schema into whatever format the model was trained on, here a declaration between tool tokens with the description and parameters.

Tool call out

The engine also converts the model's tool call into a parsed output, so the developer receives the tool name and arguments directly.

TOOL_DECLARATION

```
declaration: multiply {
  description: <|"|> Multiply two values. <|"|>,
  parameters: {
    properties:{
      a:   {type:<|"|>STRING<|"|>},
      b:   {type:<|"|>STRING<|"|>}
    },
    required: [<|"|> a <|"|>,<|"|> b <|"|>],
    type: <|"|>OBJECT<|"|>
  }
}
```

Grootendorst & Alammar (2026). Fig. 5-26, PDF p. 279

Native tool calling in the TinyAgent

Schema from a function

A helper converts any Python function into an OpenAI-style schema by reading its name, its documentation, and its parameters.

NativeTools class

It creates the schemas, needs no prompt text because the engine handles that, parses the model's tool call into the agent's format, and uses the tool role for observations.

Same usage

Usage is unchanged: register tools, give them to the agent, and ask the same question. The agent is done when there is no tool call.

Result

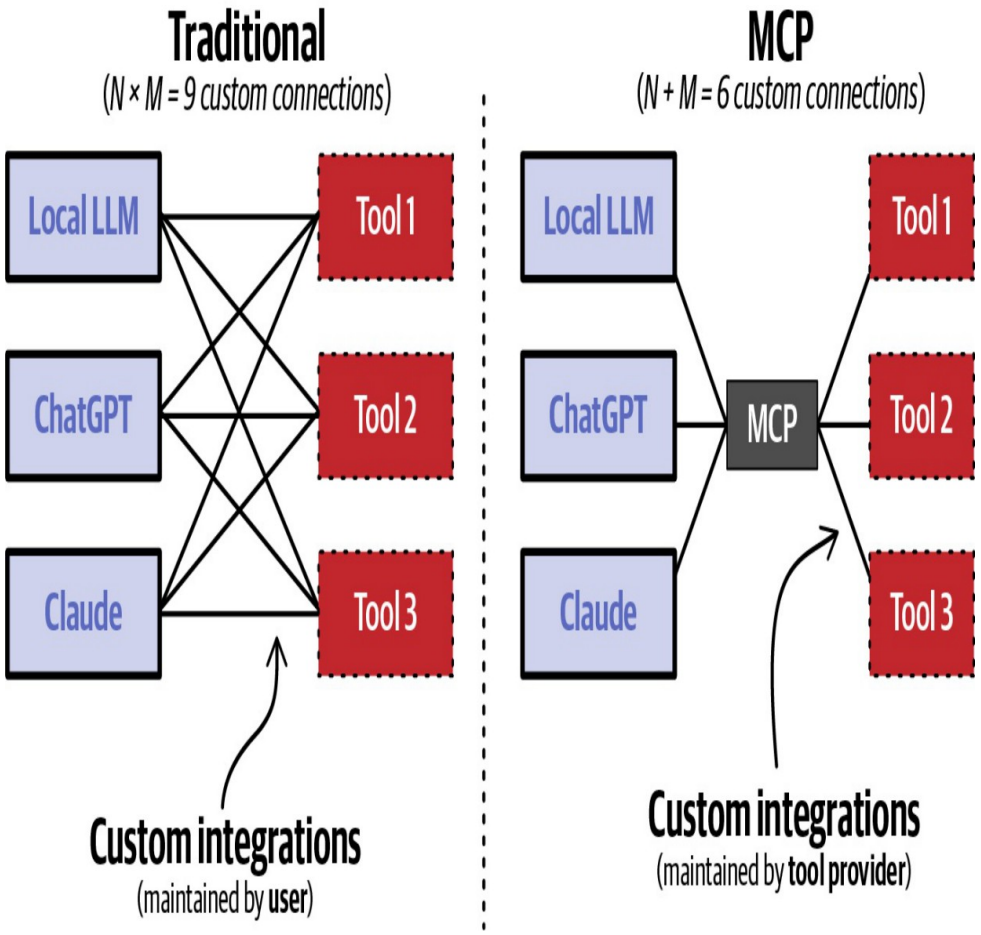
The stored messages show an assistant message with tool calls naming multiply and its arguments, followed by a tool message with 37.23.

MCP solves the N × M integration problem

The problem
When a new LLM arrives with its own way of calling tools, every tool integration has to be rewritten. With N models and M tools that means N × M custom connections.

The protocol
The Model Context Protocol is an open standard released by Anthropic in November 2024. It is often called the USB-C port of AI.

N + M
With MCP each model and each tool connects once to the protocol, so N + M connections are maintained. Maintenance moves to the tool provider.



Grootendorst & Alammar (2026). Fig. 5-27, PDF p. 284

Why manual tool descriptions do not scale

Manual work

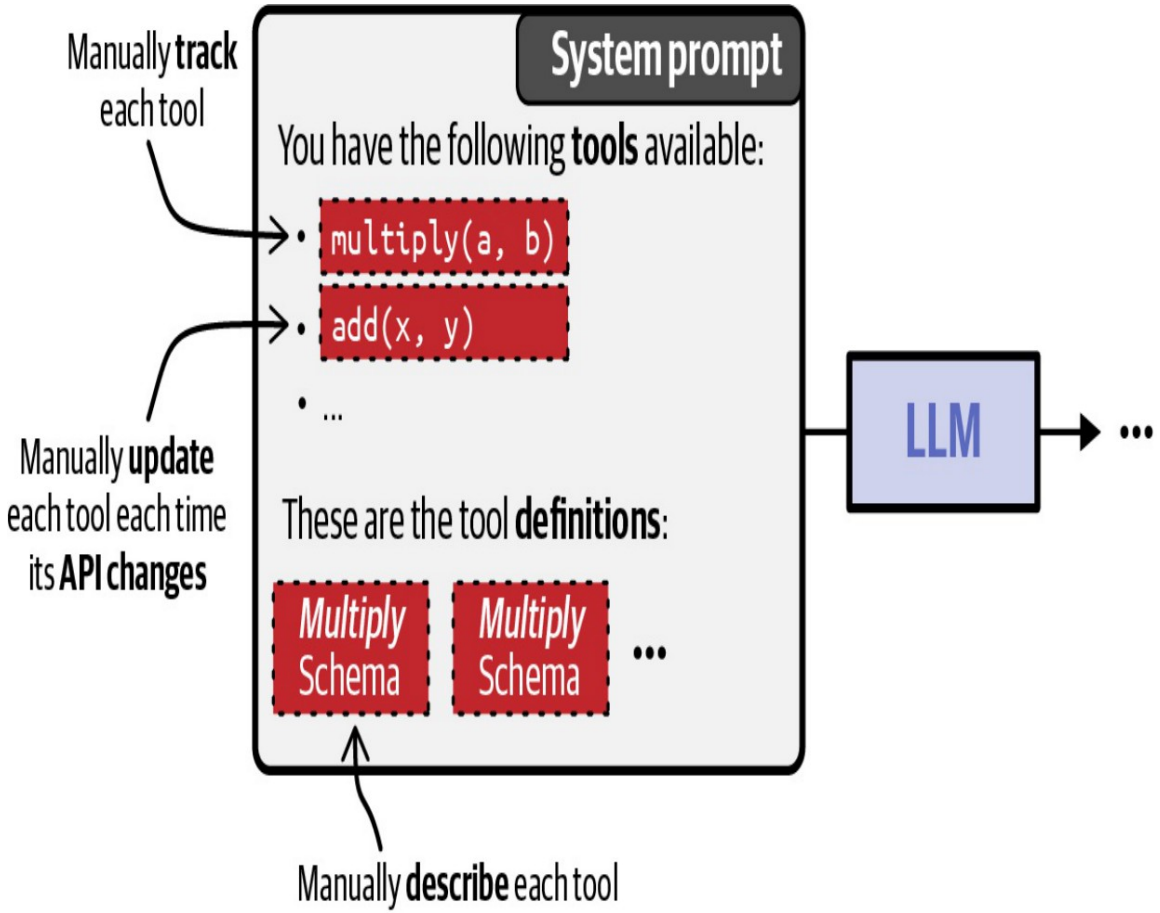
Without MCP, every tool must be tracked and fed to the LLM by hand, described with its expected JSON schema, and updated whenever its API changes.

Standardization

MCP addresses both the N × M problem and the lack of a standard way to describe tools.

Other protocols

MCP is not the only protocol. Agent2Agent (A2A) standardizes communication between agents. In 2026 MCP is arguably one of the most popular, with servers for Figma, GitHub, and Home Assistant.



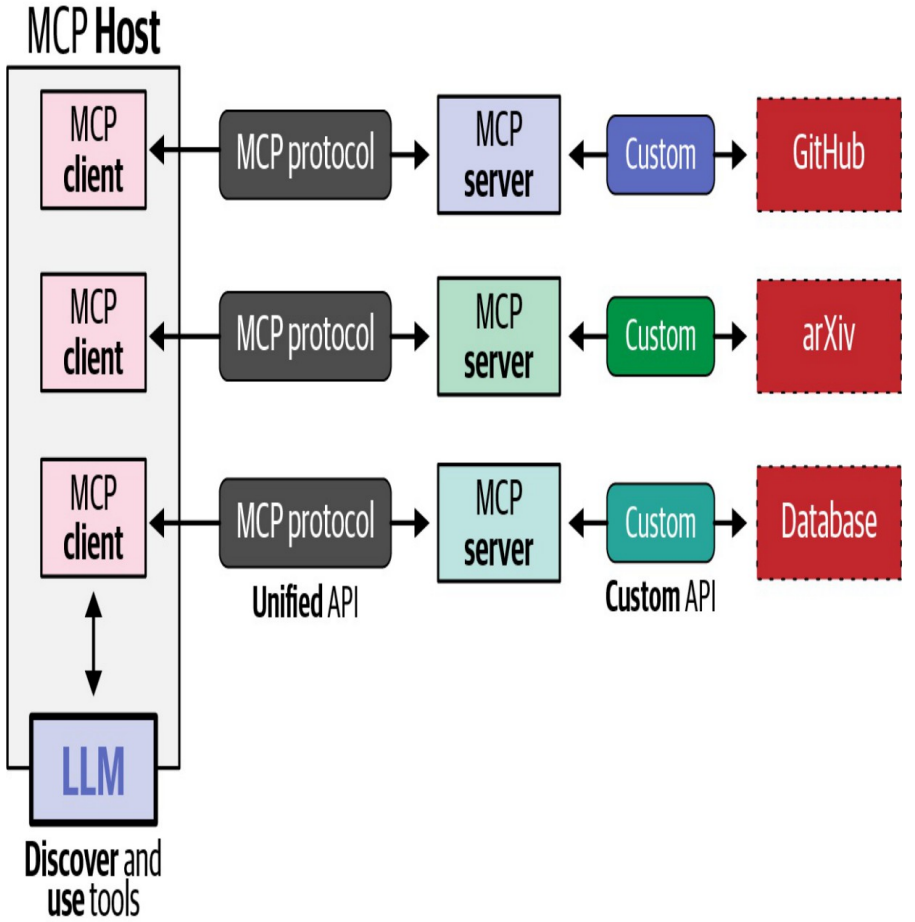
Grootendorst & Alammar (2026). Fig. 5-28, PDF p. 285

MCP has three components

MCP host
The LLM application, such as Cursor, that manages connections, interprets tool schemas, and manages routing. Chat assistants like ChatGPT and Claude, and IDE extensions, are typical hosts.

MCP client
Lives inside the host and keeps a one-to-one connection with each server, handling discovery of tools, request forwarding, and similar tasks.

MCP server
A lightweight program that exposes tools, resources, and prompts for a data source or service, such as an arXiv server for searching and loading papers.



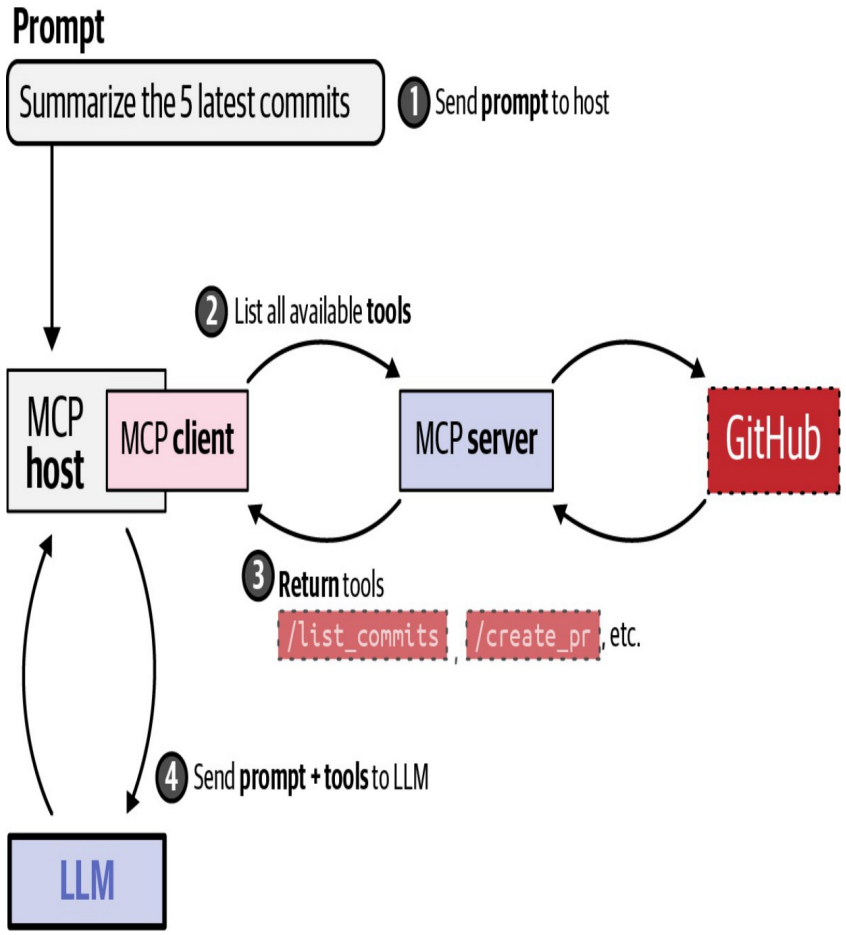
Grootendorst & Alammar (2026). Fig. 5-29, PDF p. 287

The MCP flow: discovering tools

Steps 1 to 3
The user asks to summarize the five latest commits. The host asks the server, through the client, which tools exist, and the server returns the list, such as list commits and create pull request.

Step 4
The original prompt and the list of available tools are sent to the LLM.

Discovery
The LLM does not need prior knowledge of the tools, because the server communicates what is available.



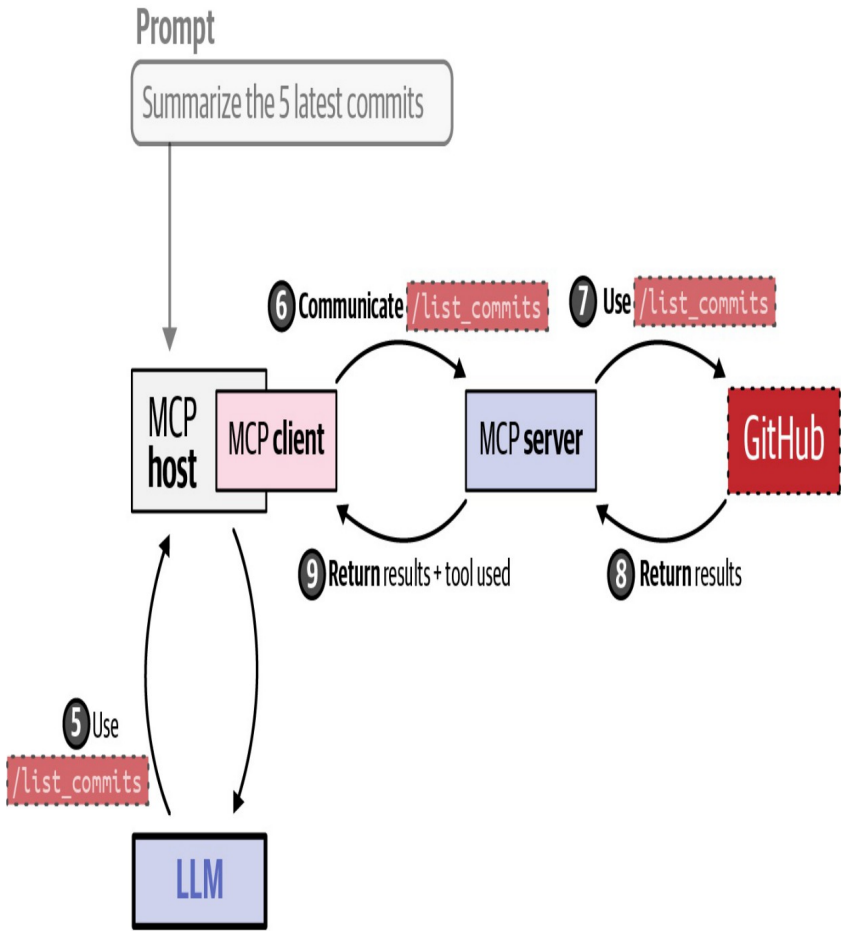
Grootendorst & Alammar (2026). Fig. 5-30, PDF p. 288

The MCP flow: choosing and executing a tool

Steps 5 to 6
Because the question is about commits, the LLM chooses the list commits tool. The client communicates that action to the server.

Steps 7 to 9
The server executes the command against GitHub. The output returns to the server and then to the client and host through MCP.

Format
The LLM must still be able to call tools, and calls follow a JSON-like structure defined by the JSON-RPC 2.0 specification, handled by the client.



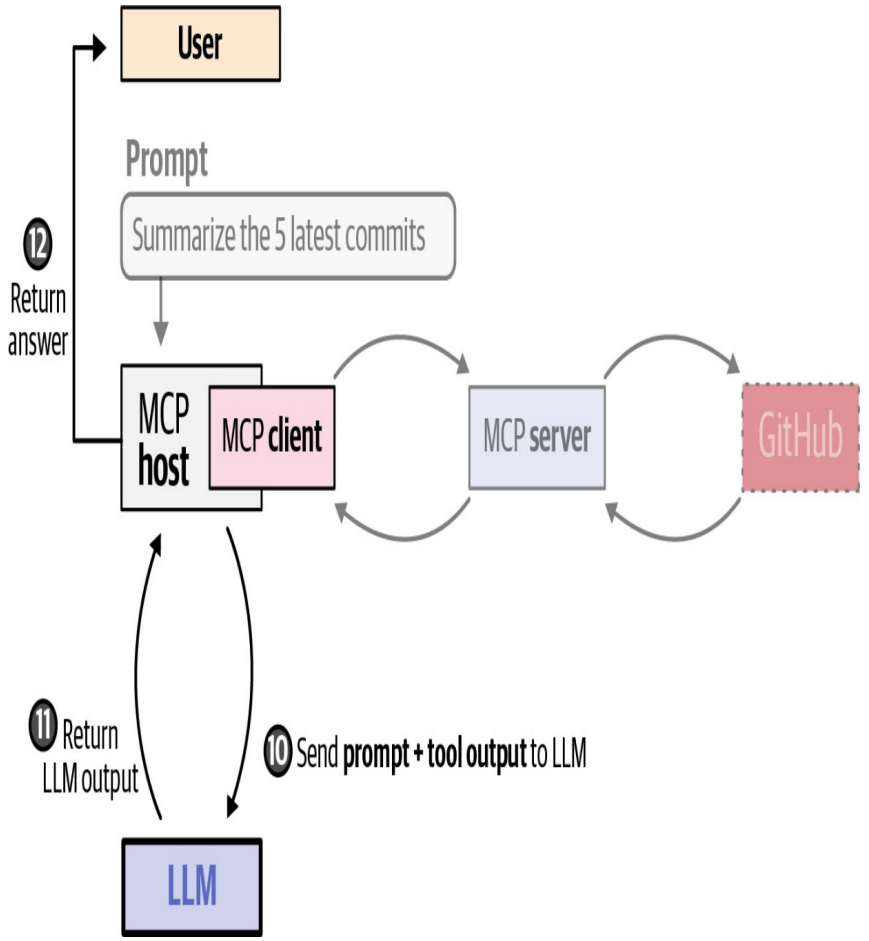
Grootendorst & Alammar (2026). Fig. 5-31, PDF p. 289

The MCP flow: using the result

Step 10
The LLM receives the results. It can choose to run another tool or to return the output.

Steps 11 and 12
In the example it summarizes the five latest commits and returns the summary to the host and then to the user.

Benefit
The LLM can discover which tools exist and choose among them, and does not have to deal with deprecated API functions.



Grootendorst & Alammar (2026). Fig. 5-32, PDF p. 290

What MCP does and does not provide

Provides a standard

A common way to list and call tools, so that a tool provider maintains one server for any compatible host and the host needs no custom code per tool.

Provides discovery

Three kinds of primitives from a server: tools, resources, and prompt templates. Discovery of what a server offers at connection time.

Does not provide skill

A model that can call tools well. The LLM must still select the right tool and format the call correctly.

Does not provide permission

Judgment about what should be allowed. Which servers to connect and which actions to approve are still design and governance decisions.

Teaching addition. Concepts: Ch. 5, PDF pp. 283–291

Skills add procedural knowledge

A gap

Tools and MCP give an agent access to actions, but not when to use them or how they fit into a larger workflow.

What a Skill is

Skills, created by Anthropic, are sets of instructions, scripts, and resources that the agent uses as context for a task. They hold procedural knowledge.

Example recipe

To create a presentation, an agent may search the web (a tool), summarize the findings (by itself), and build the slides one at a time (a tool). A Skill records this recipe.

Teach what, when, how

Skills teach the agent what to do, when to do it, and how, so users need not repeat team conventions each time, for instance to use one package tool over another.

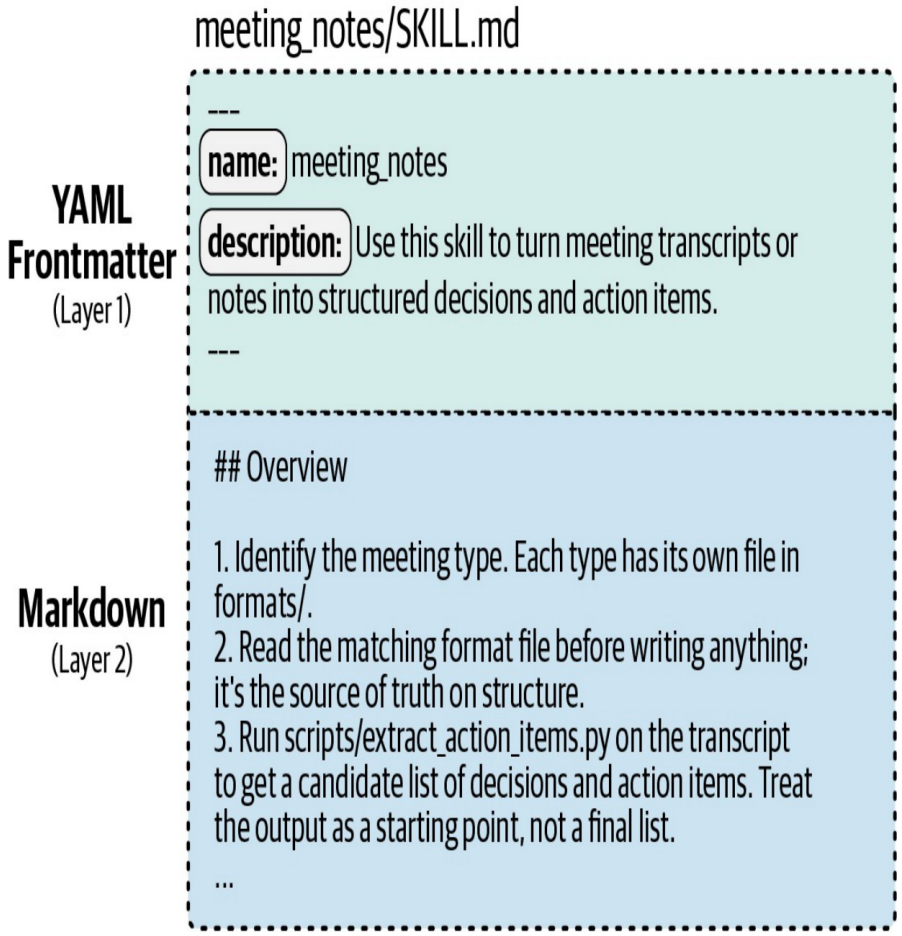
Grootendorst & Alammar (2026). PDF pp. 291–292

The SKILL.md file

Required file
Every Skill has a SKILL.md file. It starts with metadata, a name and a short description, written as YAML.

Always visible
Only the names and descriptions of all Skills are always shared with the agent, often in the system prompt, so it can decide whether a Skill is worth exploring.

Body on demand
The instructions in the body are given only when the agent asks for them. This is context engineering: long instructions that are not relevant do not use space.



Grootendorst & Alammar (2026). Fig. 5-33, PDF p. 293

Bundled resources keep Skills organized

Why bundle

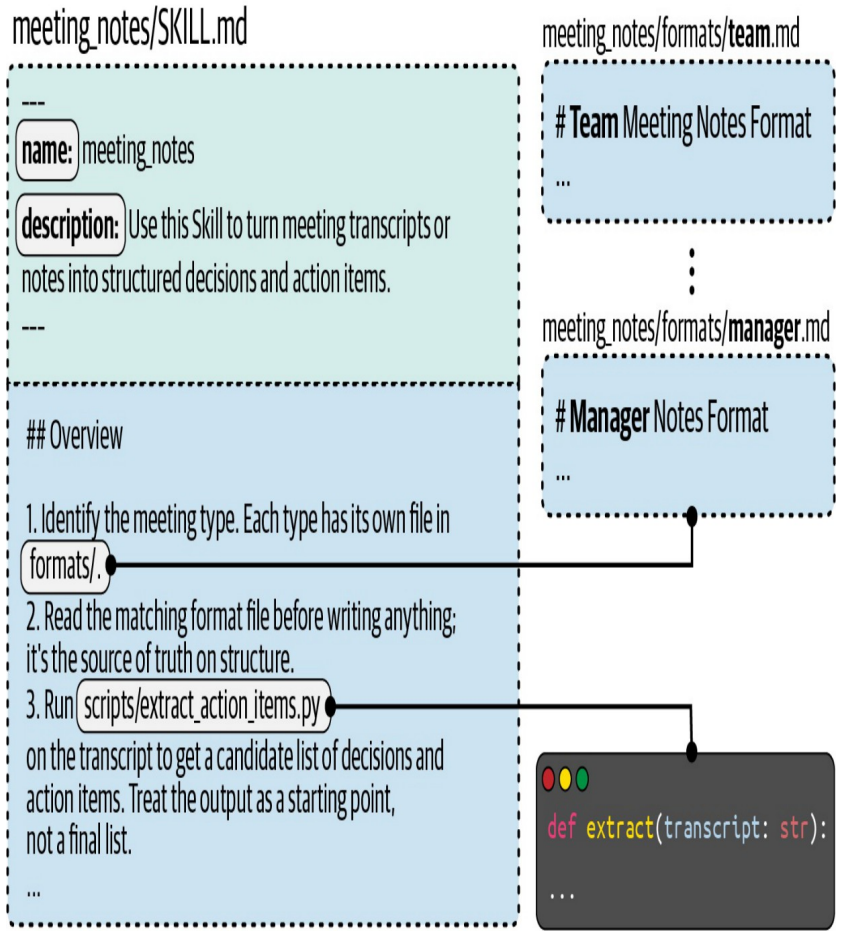
A single file grows unwieldy. Meetings differ: a one-to-one with a manager needs a different structure than a team meeting or an external call.

Extra files

Additional files, such as Python scripts and Markdown instructions for each meeting type, sit in the Skill's directory and are referenced from SKILL.md.

Loaded when needed

The agent can load or run these files only when they are needed. Only SKILL.md is required.



Grootendorst & Alammar (2026). Fig. 5-34, PDF p. 294

Progressive disclosure in three layers

Layer 1
SKILL.md metadata in YAML. Always loaded. About 100 tokens.

Layer 2
SKILL.md instructions in Markdown. Loaded when the Skill is activated. Fewer than 5,000 tokens.

Layer 3
Bundled files such as Markdown, scripts, and data. Loaded when needed. As many tokens as needed.

Why it matters
The agent starts with minimal information and gets more only as a task requires it. This keeps the amount of context used to a minimum.

Grootendorst & Alammar (2026). Table 5-1, PDF p. 295

Tools, MCP, and Skills compared

Tool

A function or API the agent can invoke. Answers what the agent can do. Defined by a name, description, and parameters.

Native tool calling

A way for a model trained on tool use to emit calls in its own format. Answers how the call is expressed.

MCP

A protocol that standardizes how tools and models connect. Answers how tools are exposed to many applications without custom work.

Skill

Instructions, scripts, and resources for a recurring task. Answers when and in what order to use tools, and how to do the job well.

Teaching addition. Concepts: Ch. 5, PDF pp. 239–296

Learning check with worked answers

1. Who actually runs a tool?
Not the model. The LLM outputs text that expresses an intention. External software parses the call, executes the function, and returns the result.

2. Why can too many tools hurt?
Each definition uses context and makes selection harder. Keep the set small, keep each tool narrow, and retrieve definitions when there are many.

3. Why is RL suited to tool use?
Tool calls are easy to verify, for example the right name and arguments. That gives a reliable reward, and RL tends to generalize better than imitation.

4. What problem does MCP solve?
The N × M problem. Each model and tool connects once to a standard, so N + M connections are maintained and providers handle API changes.

Teaching addition. Concepts: Ch. 5. Cross-reference: Ch. 4, 6, 10

Core distinctions and source guide

Conceptual distinctions

A tool is a function or API. The LLM expresses an intention, and external software executes it. A definition tells the model what a tool is, selection chooses one, and calling issues it. Learning builds tool use into the model, MCP standardizes how tools are connected, and Skills supply the procedure for a task.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 5. Selected connections: Chapters 2, 3, 4, 6, and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF